

Backend Technical Challenge

Payment Flow API · Node.js

Rol objetivo	Backend Developer
Stack requerido	Node.js · Express · Base de datos local
Duración estimada	4 – 8 horas
Entrega	Repositorio Git (GitHub)
Versión	1.0 — Mayo 2026

1. Objetivo

Implementar una pequeña API REST que simule un flujo de pago end-to-end. El candidato debe demostrar dominio en diseño de endpoints, validación de datos financieros, manejo de estados y persistencia local.

Gracias. 2. Requerimientos Funcionales

2.1 Servicio de Autenticación — POST /auth/token

Genera un Bearer Token que debe ser enviado en todos los requests posteriores.

Request body

	Descripción
client_id	Identificador del cliente (string)
client_secret	Secreto del cliente (string)

Response exitoso (200)

```
{
  "access_token": "<jwt_o_token_opaco>",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Reglas de negocio

- Credenciales inválidas → 401 Unauthorized.
- El token expira en 1 hora (configurable vía variable de entorno).
- Credenciales válidas para el challenge (hardcoded o en .env):
 - client_id: payment_client / client_secret: s3cr3t_key

2.2 Servicio de Pago — POST /payments/charge

Procesa un cargo con tarjeta. Requiere Bearer Token válido en el header Authorization.

Request body

Campo	Tipo	Descripción
reference	string	Referencia única del pago. Longitud mínima 6 chars.
amount	number	Monto en USD. Debe ser > 0. Precision: 2 decimales.
card.number	string	PAN de la tarjeta. Solo dígitos.
card.holder	string	Nombre del titular.
card.expiry	string	Fecha de vencimiento en formato MM/YY.
card.cvv	string	Código de seguridad. 3 dígitos (Visa/MC) o 4 (Amex).
currency	string	ISO 4217. Ej: "USD". Default: "USD".

Response exitoso (200)

```
{
  "transaction_id": "<uuid>",
  "reference":      "<referencia_enviada>",
  "status":        "approved" | "declined",
  "decline_reason": null | "<motivo>",
  "card_brand":    "visa" | "mastercard" | "amex",
  "card_number":   "<primeros_6_dígitos> **** <4_últimos_dígitos>",
  "amount":        10.00,
  "currency":      "USD",
  "created_at":    "<ISO 8601>"
}
```

3. Reglas de Negocio del Pago

3.1 Tarjeta aprobada (whitelist)

La siguiente tarjeta debe SIEMPRE retornar status "approved" sin importar el monto (excepto si viola otras reglas):

Tarjeta aprobada configurada

Número : 4111 1111 1111 1111

Holder : TEST USER

Expiry : 12/29

CVV : 123

Brand : Visa

Esta tarjeta puede ser definida en variables de entorno o en un archivo de configuración. No hardcodear directamente en la lógica de negocio.

3.2 Regla de monto

Rango de monto (USD)	Resultado	decline_reason
$10.00 \leq \text{amount} \leq 100.00$	approved	null
$\text{amount} < 10.00$	declined	"AMOUNT_BELOW_MINIMUM"
$\text{amount} > 100.00$	declined	"AMOUNT_EXCEEDS_LIMIT"

Nota: La tarjeta aprobada (whitelist) sigue sujeta a esta regla de monto.

3.3 Detección de red / brand

Brand	Prefijos	Longitud PAN
Visa	4	13 o 16 dígitos
Mastercard	51-55 / 2221-2720	16 dígitos
Amex	34 / 37	15 dígitos
Desconocida	Cualquier otro	Declinar: "UNSUPPORTED_CARD_BRAND"

3.4 Validación de tarjeta

3.4.1 Longitud del PAN

- Visa: 13 o 16 dígitos. Fuera de rango → declined: "INVALID_CARD_NUMBER".
- Mastercard: exactamente 16 dígitos. Fuera de rango → declined: "INVALID_CARD_NUMBER".
- Amex: exactamente 15 dígitos. Fuera de rango → declined: "INVALID_CARD_NUMBER".
- El número solo puede contener dígitos (sin espacios ni guiones al momento de la validación).

3.4.2 Fecha de vencimiento

- Formato esperado: MM/YY.
- Mes válido: 01 – 12.
- La tarjeta debe ser válida al final del mes de vencimiento.
- Ejemplo: si hoy es Mayo 2026, una tarjeta con expiry "05/26" AÚN es válida; "04/26" ya está vencida.
- Tarjeta vencida → declined: "CARD_EXPIRED".

3.4.3 CVV

- Visa / Mastercard: 3 dígitos numéricos.
- Amex: 4 dígitos numéricos.
- CVV inválido → 400 Bad Request (error de validación de entrada, no de negocio).

3.5 Detección de duplicados

Regla de duplicado

Si llega un request con una "reference" que ya existe en la base de datos (independientemente del status previo), el pago debe retornar:

status: "declined"

decline_reason: "DUPLICATE_REFERENCE"

El registro duplicado NO debe persistirse como una nueva transacción.

4. Códigos de Error HTTP

HTTP Code	Escenario	Ejemplo de body
400	Validación de entrada fallida	<code>{ "error": "VALIDATION_ERROR", "message": "..."</code>
401	Token inválido / expirado / ausente	<code>{ "error": "UNAUTHORIZED" }</code>
409	Referencia duplicada (opcional)	Puede también retornarse como 200 declined
422	Regla de negocio (monto, expirada)	<code>{ "error": "UNPROCESSABLE", "message": "..."</code>
500	Error interno no controlado	<code>{ "error": "INTERNAL_ERROR" }</code>

El candidato puede optar por retornar duplicados como 200 con status "declined" o como 409. Ambos son aceptables si la decisión es documentada.

5. Persistencia — Base de Datos Local

La base de datos puede ser cualquier solución local. Se sugieren las siguientes opciones (en orden de preferencia):

Opción	Ventaja	Consideración
SQLite + Sequelize / Knex	Sin infraestructura, SQL real	Archivo .db en el repo (gitignore)
MongoDB local (mongod)	Más cercano a producción	Requiere MongoDB instalado
lowdb / nedb	Zero config, JSON/file	Solo para challenge, no producción
PostgreSQL local	Producción-grade	Debe documentar setup

5.1 Schema mínimo requerido — Tabla / Colección: transactions

Campo	Tipo	Notas
transaction_id	UUID / string	PK generado en el servidor
reference	string	INDEX único para detección de duplicados
status	string	"approved" "declined"
decline_reason	string null	null si approved
amount	decimal / number	2 decimales
currency	string	ISO 4217
card_brand	string	"visa" "mastercard" "amex"
card_number	string	Primeros 5 y últimos 4 dígitos de la tarjeta
card_holder	string	Nombre del titular
created_at	datetime	ISO 8601 UTC

IMPORTANTE: El PAN completo NO debe persistirse. Solo first_six y last_four.

6. Casos de Prueba Esperados

El candidato debe incluir una colección de Postman / Insomnia o un archivo de tests con al menos los siguientes escenarios:

#	Escenario	Card Number	Amount	Resultado esperado
1	Pago exitoso – whitelist	4111111111111111	\$5.00	approved
2	Monto mínimo borde inferior	4111111111111111	\$1.00	approved
3	Monto máximo borde superior	4111111111111111	\$10.00	approved
4	Monto bajo el mínimo	4111111111111111	\$0.99	declined: AMOUNT_BELOW_MINIMUM
5	Monto sobre el límite	4111111111111111	\$10.01	declined: AMOUNT_EXCEEDS_LIMIT
6	Referencia duplicada	4111111111111111	\$5.00	declined: DUPLICATE_REFERENCE
7	Tarjeta Mastercard válida	5500005555555559	\$5.00	declined o approved*
8	Tarjeta Amex válida	378282246310005	\$5.00	declined o approved*
9	Tarjeta vencida	4111111111111111	\$5.00	declined: CARD_EXPIRED
10	PAN longitud inválida	41111111111	\$5.00	declined: INVALID_CARD_NUMBER
11	Token ausente / inválido	4111111111111111	\$5.00	401 Unauthorized
12	Brand no soportada	6011000990139424	\$5.00	declined: UNSUPPORTED_CARD_BRAND

** Para tarjetas distintas a la whitelist con monto válido, el resultado puede ser approved o declined según la lógica implementada por el candidato (puede simular aprobación aleatoria o siempre declinar fuera de la whitelist). Lo que importa es que la detección de brand y las validaciones sean correctas.*

7. Estructura de Proyecto Sugerida

El candidato es libre de organizar el proyecto como prefiera. Esta es una estructura de referencia:

```
payment-challenge/
├── src/
│   ├── config/           # Variables de entorno, constantes
│   ├── middleware/       # Auth middleware (verify bearer token)
│   ├── routes/           # auth.routes.js, payment.routes.js
│   ├── controllers/      # auth.controller.js, payment.controller.js
│   ├── services/         # token.service.js, payment.service.js
│   ├── validators/       # card.validator.js, payment.validator.js
│   └── db/               # db.js (conexión), models/, migrations/
├── tests/                # Unit & integration tests
├── postman/              # Collection exportada (o Insomnia)
├── .env.example
├── README.md
└── package.json
```

8. Variables de Entorno (.env.example)

```
# Server
PORT=3000
NODE_ENV=development

# Auth
JWT_SECRET=super_secret_jwt_key
TOKEN_EXPIRY_SECONDS=3600
VALID_CLIENT_ID=payment_client
VALID_CLIENT_SECRET=s3cr3t_key

# Whitelist card
APPROVED_CARD_NUMBER=4111111111111111

# Database
DB_PATH=./data/payments.db # para SQLite
```


9. Criterios de Evaluación

Criterio	Peso	Descripción
Funcionalidad completa	30%	Todos los casos de prueba pasan correctamente
Validaciones y reglas de negocio	25%	Detección de brand, longitud, vencimiento, duplicados, monto
Calidad y organización del código	20%	Separación de concerns, naming, SOLID básico
Manejo de errores	15%	HTTP codes correctos, mensajes claros, no exposición de stack traces
Documentación y README	10%	Instrucciones de setup claras, colección Postman o tests

10. Puntos Bonus (Opcionales)

- Tests unitarios con Jest / Mocha (cobertura > 70%).
- Middleware de rate limiting en /payments/charge.
- Logging estructurado con Winston o Pino.
- Docker / docker-compose para levantar el proyecto con un solo comando.
- Validación de Luhn para el PAN de la tarjeta.
- Endpoint GET /payments/:transaction_id para consultar una transacción.
- Paginación en GET /payments (listado de transacciones).

11. Entregables

1. Repositorio Git con historial de commits legible (no un solo commit "init").
2. Archivo README.md con:
 - Instrucciones de instalación y ejecución.
 - Variables de entorno requeridas.
 - Decisiones de diseño tomadas (BD elegida, manejo de duplicados, etc.).
3. Colección Postman / Insomnia exportada (o suite de tests ejecutable).
4. Archivo .env.example (nunca subir .env con valores reales).

12. Notas Finales

Libertad de diseño

El candidato tiene libertad para elegir: framework HTTP (Express, Fastify, Hapi, Koa), ORM/ODM, base de datos local, y estructura de carpetas. Lo que se evalúa es el criterio técnico detrás de cada decisión, no el uso de una tecnología específica.

Seguridad básica

Aunque es un ambiente de challenge, se espera que el candidato demuestre consciencia de buenas prácticas: no persistir el PAN completo, no exponer el JWT_SECRET, no loggear datos sensibles de tarjeta en consola.

Cualquier duda sobre los requerimientos debe quedar documentada en el README con la decisión tomada por el candidato. No hay respuestas incorrectas si la decisión es razonable y justificada.